



Trainee SE Technical Assessment

Internal Inventory & Audit Management System

1. Overview

You are required to design and implement a full-stack Internal Equipment Management System. This system will allow employees to manage assigned equipment while providing administrators full visibility and control over company inventory.

2. Objectives

- Build a secure, scalable full-stack application
- Implement role-based access control
- Demonstrate backend API design and frontend integration
- Follow clean architecture and best practices

3. Functional Requirements

3.1 Authentication & Authorization

- Implement user registration and login
- Use JWT-based authentication
- Roles: Employee, Admin
- Restrict access based on roles

3.2 Employee Capabilities

- Create equipment records
- View only their own equipment
- Fields: Device Name, Serial Number, Assigned Date, Status



3.3 Admin Capabilities

- View all equipment records
- Update equipment status (Active, Damaged, Retired)
- Delete records

4. API Design (Expected Endpoints)

Below is a **suggested set of core endpoints** to guide your implementation. You are **not restricted to this structure** and are encouraged to design additional endpoints or modify these based on your architectural decisions.

POST /api/auth/register

POST /api/auth/login

GET /api/equipment (Employee: own, Admin: all)

POST /api/equipment

PUT /api/equipment/:id (Admin only)

DELETE /api/equipment/:id (Admin only)

5. Data Modeling

Below are **suggested core data models** to guide your implementation. You are **not restricted to these structures** and are encouraged to extend or refine them based on your design decisions.

User Schema:

- name, email, password (hashed), role

Equipment Schema:

- deviceName, serialNumber, assignedDate, status, assignedTo (userId)

6. Security Requirements (CRITICAL)

- Passwords must be hashed using bcrypt
- No plain text passwords
- JWT must be validated on every request
- Middleware must enforce role-based access



- Prevent unauthorized access via tools like Postman
- Validate inputs to prevent invalid data

7. Testing Requirements

- Write backend unit tests (Jest/Mocha)
- Test authentication flow
- Test protected endpoints
- Handle error cases properly

8. Frontend Requirements

- React-based UI
- Separate dashboards for Employee/Admin
- Handle loading states and unauthorized access
- Prevent UI flash of restricted content

9. Code Quality Expectations

- Use MVC or layered architecture
- Separate routes, controllers, services
- Use environment variables
- Maintain clean and readable code

10. Bonus (Optional)

- Add pagination/filtering
- Deploy application (Vercel/Render/AWS)
- Implement audit logs



11. Deadline & Submission Process

Deadline

- You are expected to complete and submit the assignment within **5–7 days** from the date of receiving this assessment.
- If you require additional time due to valid reasons, you may request an extension in advance.

Submission Instructions

- Submit your completed assignment by **replying to the same email** in which this assessment was shared.
- Include the following in your response:
 - A **GitHub repository link** (public or with access granted)
 - Any additional notes or instructions required to run your project
 - A short explanation of your approach, design decisions, or trade-offs

Important Notes

- Ensure your repository is accessible before submission
- Incomplete or non-runnable projects may affect evaluation
- Clear documentation will be considered as part of the assessment



12. Evaluation Criteria (Internal Use Only)

Category	Criteria	Expectation	Marks
Security	Password Handling	bcrypt used, no plain text	
Security	API Protection	JWT + RBAC enforced	
Backend	API Design	RESTful and structured	
Backend	Data Validation	Mongoose validation used	
Frontend	UI/UX	Role-based rendering, clean UI	
Functionality	CRUD Operations	All features working	
Testing	Unit/API Tests	Core flows tested	
Architecture	Code Structure	Clean separation of concerns	
Docs	README	Clear setup instructions	
Bonus	Advanced Features	Deployment, Docker, etc.	

* AI Usage & Ownership Disclaimer

The use of AI tools, including advanced and agentic AI platforms, to assist in building this assignment is **permitted and encouraged** as part of modern software engineering practices.

There are **no restrictions** on how you choose to leverage such tools during development.

However, it is essential that:

- **You retain full ownership of the submitted solution.**
- You have a clear understanding of all implemented features, logic, and design decisions.
- You are able to confidently **explain, modify, and extend your code** during the interview process.
- You take full responsibility for the **correctness, security, and quality** of your implementation.

Please note that your **technical depth and understanding will be rigorously evaluated**, and inability to demonstrate ownership of your work may negatively impact your assessment.